

Section 1: Setup

This section focuses on importing the necessary libraries and configuring the environment for the project.

Cell 1 & 2: Markdown

1 # 1. Setup

2 ## 1.1 Import Dependencies

These are titles indicating the start of the setup process and the subsection for importing libraries.

Cell 3: Code - Standard Dependencies

1 # Import standard dependencies

2 import cv2

3 import os

4 import random

5 import numpy as np

6 from matplotlib import pyplot as plt

- import cv2: Imports the OpenCV library, used for computer vision tasks like accessing the webcam and basic image manipulation.

- import os: Imports the operating system module, used for interacting with the file system, like creating directories and managing file paths.

- import random: Imports the random module, which is used here for generating random numbers (though not explicitly used in the provided code, it's a common import for data shuffling).

- import numpy as np: Imports the NumPy library with the standard alias np. NumPy is essential for efficient numerical computations, especially with multi-dimensional arrays (tensors).

- from matplotlib import pyplot as plt: Imports the pyplot interface from Matplotlib with the alias plt. It's used for plotting and visualizing data, such as displaying images.

Cell 4: Code - TensorFlow Dependencies

1 # Import tensorflow dependencies - Functional API

2 from tensorflow.keras.models import Model

3 from tensorflow.keras.layers import Layer, Conv2D, Dense, MaxPooling2D, Input, Flatten

4 import tensorflow as tf

- from tensorflow.keras.models import Model: Imports the Model class from TensorFlow's Keras API, which is used to define a neural network model.

- from tensorflow.keras.layers import ...: Imports specific layer types needed to build the convolutional neural network (CNN):

- Layer: The base class for creating custom layers.

- Conv2D: A 2D convolution layer, the core building block of a CNN for feature extraction.

- Dense: A standard, fully-connected neural network layer.

- MaxPooling2D: A layer that down-samples the feature maps, reducing their dimensions.

- Input: Used to define the entry point of the model.

- Flatten: A layer that converts a multi-dimensional tensor into a one-dimensional vector.

- import tensorflow as tf: Imports the main TensorFlow library with the standard alias tf.

Cell 5 & 6: Code - Set GPU Growth

1 ## 1.2 Set GPU Growth

```
1 # Avoid OOM errors by setting GPU Memory Consumption Growth
2 gpus = tf.config.experimental.list_physical_devices('GPU')
3 for gpu in gpus:
4     tf.config.experimental.set_memory_growth(gpu, True)
```

- This section configures how TensorFlow utilizes the GPU.
- `gpus = tf.config.experimental.list_physical_devices('GPU')`: Retrieves a list of all available GPU devices on the system.
- `for gpu in gpus::` Iterates through each found GPU.
- `tf.config.experimental.set_memory_growth(gpu, True)`: This is a crucial command that tells TensorFlow to allocate GPU memory on an as-needed basis, rather than reserving the entire memory at the start. This helps prevent "Out of Memory" (OOM) errors.

Cell 7 & 8: Code - Create Folder Structures

1 ## 1.3 Create Folder Structures

```
1 # Setup paths
2 POS_PATH = os.path.join('data', 'positive')
3 NEG_PATH = os.path.join('data', 'negative')
4 ANC_PATH = os.path.join('data', 'anchor')
```

- These lines define the file paths for storing the different types of images needed for the Siamese network.
- `POS_PATH`: The path for "positive" images—other pictures of the person you want to recognize.
- `NEG_PATH`: The path for "negative" images—pictures of different people.
- `ANC_PATH`: The path for "anchor" images—the baseline pictures of the person you want to recognize.

Cell 9: Code - Make Directories

```
1 # Make the directories
2 os.makedirs(POS_PATH)
3 os.makedirs(NEG_PATH)
4 os.makedirs(ANC_PATH)
```

- `os.makedirs(...)`: This function creates the actual directories on the file system based on the paths defined in the previous cell. It will also create the parent data directory if it doesn't already exist.

Section 2: Collect Positives and Anchors

This section deals with gathering the image data. The "negative" images are taken from a standard dataset, while the "anchor" and "positive" images are collected from the user's webcam.

Cell 10-13: Code - Untar LFW Dataset

1 # 2. Collect Positives and Anchors 2 ## 2.1 Untar Labelled Faces in the Wild Dataset

```
1 # http://vis-www.cs.umass.edu/lfw/
2 # Uncompress Tar GZ Labelled Faces in the Wild Dataset
3 !tar -xf lfw.tgz
```

- The markdown provides a title for the section and a link to the "Labelled Faces in the Wild" (LFW) dataset, a common resource for face recognition research.

- !tar -xf lfw.tgz: This is a shell command run from within the notebook (indicated by the !). It extracts the contents of the lfw.tgz archive, which contains the negative images.

Cell 14: Code - Move LFW Images

```
1 # Move LFW Images to the following repository data/negative
2 for directory in os.listdir('lfw'):
3     for file in os.listdir(os.path.join('lfw', directory)):
4         EX_PATH = os.path.join('lfw', directory, file)
5         NEW_PATH = os.path.join(NEG_PATH, file)
6         os.replace(EX_PATH, NEW_PATH)
```

- This code block iterates through all the extracted LFW image files, which are organized in subdirectories, and moves them into the data/negative folder. This populates our set of negative examples.

Cell 15-17: Code - Prepare for Image Collection

```
1 ## 2.2 Collect Positive and Anchor Classes
```

```
1 # Import uuid library to generate unique image names
2 import uuid
3 os.path.join(ANC_PATH, '{}.jpg'.format(uuid.uuid1()))
```

- import uuid: Imports the uuid library to generate unique identifiers, which are used to create unique filenames for each captured image.

- The os.path.join line is a demonstration of how a unique file path will be constructed.

Cell 18: Code - Webcam Image Collection

```
1 # Establish a connection to the webcam
2 cap = cv2.VideoCapture(4)
3 while cap.isOpened():
4     ret, frame = cap.read()
5     # Cut down frame to 250x250px
6     frame = frame[120:120+250, 200:200+250, :]
7     # Collect anchors
8     if cv2.waitKey(1) & 0xFF == ord('a'):
9         imgname = os.path.join(ANC_PATH, '{}.jpg'.format(uuid.uuid1()))
10        cv2.imwrite(imgname, frame)
11    # Collect positives
12    if cv2.waitKey(1) & 0xFF == ord('p'):
13        imgname = os.path.join(POS_PATH, '{}.jpg'.format(uuid.uuid1()))
14        cv2.imwrite(imgname, frame)
15    # Show image back to screen
16    cv2.imshow('Image Collection', frame)
17    # Breaking gracefully
18    if cv2.waitKey(1) & 0xFF == ord('q'):
19        break
```

```

20 # Release the webcam
21 cap.release()
22 # Close the image show frame
23 cv2.destroyAllWindows()

```

- This is an interactive script to collect anchor and positive images.
- `cap = cv2.VideoCapture(4)`: Opens a connection to the webcam (device index 4; this might be 0 or 1 on other systems).
- `while cap.isOpened()`: The main loop that runs as long as the webcam is active.
- `ret, frame = cap.read()`: Reads one image (a frame) from the webcam.
- `frame = frame[...]`: Crops the captured frame to a 250x250 pixel region.
- The `if cv2.waitKey(1) ...` blocks check for keyboard input:
 - Pressing 'a' saves the current frame as an anchor image.
 - Pressing 'p' saves the current frame as a positive image.
 - Pressing 'q' breaks the loop and ends the script.
- `cv2.imshow(...)`: Displays the live webcam feed in a window.
- `cap.release()` and `cv2.destroyAllWindows()`: Properly closes the webcam and all OpenCV windows.

Cell 19: Code - Display a Frame

```

1 plt.imshow(frame[120:120+250,200:200+250, :])

```

- This line uses Matplotlib to display the last captured frame, which is useful for verification.

Section 3: Load and Preprocess Images

This section prepares the collected image data to be fed into the neural network. This involves creating TensorFlow Dataset objects and applying preprocessing steps.

Cell 20 & 21: Code - Get Image Directories

```

1 # 3. Load and Preprocess Images
2 ## 3.1 Get Image Directories

```

```

1 anchor = tf.data.Dataset.list_files(ANC_PATH+'\\*.jpg').take(300)
2 positive = tf.data.Dataset.list_files(POS_PATH+'\\*.jpg').take(300)
3 negative = tf.data.Dataset.list_files(NEG_PATH+'\\*.jpg').take(300)

```

- `tf.data.Dataset.list_files(...)`: Creates a TensorFlow Dataset object containing the file paths of all .jpg images in the specified directories.
- `.take(300)`: Limits the dataset to 300 images for each category (anchor, positive, and negative).

Cell 22-24: Code - Test the Datasets

```

1 dir_test = anchor.as_numpy_iterator()
2 print(dir_test.next())

```

- These lines are for inspecting the dataset. `as_numpy_iterator()` creates an iterator that yields file paths, and `next()` retrieves the first one.

Cell 25 & 26: Code - Preprocessing Function

1 ## 3.2 Preprocessing - Scale and Resize

```

1 def preprocess(file_path):
2     # Read in image from file path

```

```

3 byte_img = tf.io.read_file(file_path)
4 # Load in the image
5 img = tf.io.decode_jpeg(byte_img)
6 # Preprocessing steps - resizing the image to be 100x100x3
7 img = tf.image.resize(img, (100,100))
8 # Scale image to be between 0 and 1
9 img = img / 255.0
10 # Return image
11 return img

```

- This defines a crucial function preprocess that will be applied to every image.
- `tf.io.read_file`: Reads the image file from its path.
- `tf.io.decode_jpeg`: Decodes the raw file into an image tensor.
- `tf.image.resize(img, (100,100))`: Resizes every image to 100x100 pixels. This ensures all inputs to the model have a uniform size.
- `img = img / 255.0`: Normalizes the pixel values from the range [0, 255] to [0, 1]. This is a standard practice that helps the model train more effectively.

Cell 27-30: Code - Test Preprocessing

```

1 img = preprocess('data\\anchor\\a4e73462-135f-11ec-9e6e-a0cec8d2d278.jpg')
2 img.numpy().max()
3 plt.imshow(img)
4 dataset.map(preprocess)

```

- These lines test the preprocess function on a single image and display the result to visually confirm it works as expected. The final line is incomplete as dataset is not yet defined.

Cell 31-33: Code - Create Labelled Dataset

```

1 ## 3.3 Create Labelled Dataset

1 # (anchor, positive) => 1,1,1,1,1
2 # (anchor, negative) => 0,0,0,0,0
3 positives = tf.data.Dataset.zip((anchor, positive,
tf.data.Dataset.from_tensor_slices(tf.ones(len(anchor)))))
4 negatives = tf.data.Dataset.zip((anchor, negative,
tf.data.Dataset.from_tensor_slices(tf.zeros(len(anchor)))))
5 data = positives.concatenate(negatives)

```

- This is where the image pairs and their corresponding labels are created.
- A pair of (anchor, positive) images is labeled with 1 (meaning "same person").
- A pair of (anchor, negative) images is labeled with 0 (meaning "different person").
- `tf.data.Dataset.zip`: Combines the datasets of image paths with a dataset of labels (1s for positives, 0s for negatives).
- `positives.concatenate(negatives)`: Merges the positive and negative pairs into a single, unified dataset.

Cell 34-36: Code - Inspect the Labeled Data

```

1 samples = data.as_numpy_iterator()
2 example = samples.next()
3 example

```

- These lines fetch and display one sample from the newly created dataset to verify its structure: (anchor_path,

validation_path, label).

Cell 37-44: Code - Build Train and Test Partitions

1 ## 3.4 Build Train and Test Partition

```
1 # Define a function to preprocess pairs
2 def preprocess_twin(input_img, validation_img, label):
3     return(preprocess(input_img), preprocess(validation_img), label)
4 # Build dataloader pipeline
5 data = data.map(preprocess_twin)
6 data = data.cache()
7 data = data.shuffle(buffer_size=1024)
8 # Training partition
9 train_data = data.take(round(len(data)*.7))
10 train_data = train_data.batch(16)
11 train_data = train_data.prefetch(8)
12 # Testing partition
13 test_data = data.skip(round(len(data)*.7))
14 test_data = test_data.take(round(len(data)*.3))
15 test_data = test_data.batch(16)
16 test_data = test_data.prefetch(8)
```

- This is the final data preparation pipeline.

- preprocess_twin: A helper function that applies the preprocess function to both images in a pair.

- data.map(preprocess_twin): Applies this function to every pair in the dataset.

- data.cache(): Caches the preprocessed data in memory to speed up training.

- data.shuffle(buffer_size=1024): Randomly shuffles the data to prevent the model from learning any spurious order.

- The dataset is then split into training (70%) and testing (30%) sets.

- .batch(16): Groups the data into batches of 16 samples. The model will process one batch at a time.

- .prefetch(8): A performance optimization that allows the CPU to prepare the next batches while the GPU is working on the current one.

Section 4: Model Engineering

This section defines the architecture of the Siamese network.

Cell 45-58: Code - Build Embedding Layer

1 # 4. Model Engineering

2 ## 4.1 Build Embedding Layer

```
1 def make_embedding():
2     inp = Input(shape=(100,100,3), name='input_image')
3     # First block
4     c1 = Conv2D(64, (10,10), activation='relu')(inp)
5     m1 = MaxPooling2D(64, (2,2), padding='same')(c1)
6     # Second block
7     c2 = Conv2D(128, (7,7), activation='relu')(m1)
8     m2 = MaxPooling2D(64, (2,2), padding='same')(c2)
9     # Third block
10    c3 = Conv2D(128, (4,4), activation='relu')(m2)
11    m3 = MaxPooling2D(64, (2,2), padding='same')(c3)
```

```

12 # Final embedding block
13 c4 = Conv2D(256, (4,4), activation='relu')(m3)
14 f1 = Flatten()(c4)
15 d1 = Dense(4096, activation='sigmoid')(f1)
16
17 return Model(inputs=[inp], outputs=[d1], name='embedding')
18
19 embedding = make_embedding()
20 embedding.summary()

```

- This code defines the core convolutional neural network (embedding model) that takes an image and converts it into a 4096-dimensional vector (an "embedding").
- The model is a standard CNN architecture: a series of Conv2D layers (for feature extraction) and MaxPooling2D layers (for down-sampling).
- Flatten(): Converts the final 2D feature map into a 1D vector.
- Dense(4096, activation='sigmoid'): A fully connected layer that produces the final 4096-dimensional embedding.
- embedding.summary(): Prints a summary of the model's architecture.

Cell 59-62: Code - Build Distance Layer

```

1 ## 4.2 Build Distance Layer

1 # Siamese L1 Distance class
2 class L1Dist(Layer):
3     # Init method - inheritance
4     def __init__(self, **kwargs):
5         super().__init__()
6     # Magic happens here - similarity calculation
7     def call(self, input_embedding, validation_embedding):
8         return tf.math.abs(input_embedding - validation_embedding)

```

- This defines a custom Keras layer, L1Dist, to calculate the "distance" between two embeddings.
- The call method implements the L1 distance, which is the absolute difference between the two embedding vectors. A smaller distance implies the images are more similar.

Cell 63-76: Code - Make Siamese Model

```

1 ## 4.3 Make Siamese Model

1 def make_siamese_model():
2     # Anchor image input in the network
3     input_image = Input(name='input_img', shape=(100,100,3))
4     # Validation image in the network
5     validation_image = Input(name='validation_img', shape=(100,100,3))
6
7     # Combine siamese distance components
8     siamese_layer = L1Dist()
9     siamese_layer._name = 'distance'
10    distances = siamese_layer(embedding(input_image), embedding(validation_image))
11
12    # Classification layer
13    classifier = Dense(1, activation='sigmoid')(distances)
14

```

```

15 return Model(inputs=[input_image, validation_image], outputs=classifier,
name='SiameseNetwork')
16
17 siamese_model = make_siamese_model()
18 siamese_model.summary()

```

- This function assembles the final Siamese network.
- It defines two inputs: input_image and validation_image.
- It passes both inputs through the same shared `embedding` model to get their respective vector embeddings. This is the core principle of a Siamese network.
- The L1Dist layer calculates the distance between the two embeddings.
- A final Dense layer with a sigmoid activation acts as a classifier. It takes the distance vector and outputs a single value between 0 and 1, representing the probability that the two images are of the same person.
- siamese_model.summary() prints the architecture of the complete model.

Section 5: Training

This section covers setting up the training process and executing it.

Cell 77-80: Code - Setup Loss and Optimizer

```

1 # 5. Training
2 ## 5.1 Setup Loss and Optimizer
1 binary_cross_loss = tf.losses.BinaryCrossentropy()
2 opt = tf.keras.optimizers.Adam(1e-4) # 0.0001

```

- binary_cross_loss: The loss function used to measure how well the model is performing. Binary Cross-Entropy is the standard choice for binary (0/1) classification problems.
- opt: The optimizer. Adam is an efficient optimization algorithm that will update the model's weights to minimize the loss. A learning rate of 1e-4 (0.0001) is chosen.

Cell 81 & 82: Code - Establish Checkpoints

```

1 ## 5.2 Establish Checkpoints

1 checkpoint_dir = './training_checkpoints'
2 checkpoint_prefix = os.path.join(checkpoint_dir, 'ckpt')
3 checkpoint = tf.train.Checkpoint(opt=opt, siamese_model=siamese_model)

```

- This sets up checkpointing, which saves the model's state (weights and optimizer state) periodically during training.
- This allows you to resume training later or restore a trained model without starting from scratch.

Cell 83-91: Code - Build Train Step Function

```

1 ## 5.3 Build Train Step Function

1 @tf.function
2 def train_step(batch):
3     # Record all of our operations
4     with tf.GradientTape() as tape:
5         # Get anchor and positive/negative image
6         X = batch[:2]

```



```

7     # Get label
8     y = batch[2]
9     # Forward pass
10    yhat = siamese_model(X, training=True)
11    # Calculate loss
12    loss = binary_cross_loss(y, yhat)
13    # Calculate gradients
14    grad = tape.gradient(loss, siamese_model.trainable_variables)
15    # Calculate updated weights and apply to siamese model
16    opt.apply_gradients(zip(grad, siamese_model.trainable_variables))
17    # Return loss
18    return loss

```

- This function defines what happens in a single training step (for one batch).

- `@tf.function`: A decorator that compiles the Python function into a high-performance TensorFlow graph.

- with `tf.GradientTape()` as `tape`:: TensorFlow's automatic differentiation tool, which records the operations to calculate gradients later.

- `yhat = siamese_model(X, training=True)`: The "forward pass" where the model makes a prediction on the input batch X.

- `loss = binary_cross_loss(y, yhat)`: Calculates the loss between the model's prediction yhat and the true labels y.

- `grad = tape.gradient(...)`: Calculates the gradients of the loss with respect to the model's trainable weights.

- `opt.apply_gradients(...)`: Applies these gradients to the model's weights, updating them to improve performance. This

is the core of the learning process.

Cell 92-95: Code - Build and Run Training Loop

```

1 ## 5.4 Build Training Loop
2 ## 5.5 Train the model

```

```

1 def train(data, EPOCHS):
2     # Loop through epochs
3     for epoch in range(1, EPOCHS+1):
4         print('\n Epoch {}/{}'.format(epoch, EPOCHS))
5         progbar = tf.keras.utils.Progbar(len(data))
6         # Loop through each batch
7         for idx, batch in enumerate(data):
8             # Run train step here
9             train_step(batch)
10            progbar.update(idx+1)
11            # Save checkpoints
12            if epoch % 10 == 0:
13                checkpoint.save(file_prefix=checkpoint_prefix)
14
15 EPOCHS = 50
16 train(train_data, EPOCHS)

```

- `train`: A function that defines the main training loop.

- It loops for a specified number of EPOCHS. An epoch is one full pass through the entire training dataset.

- Inside the epoch loop, it iterates through each batch of the `train_data`.

- `train_step(batch)`: It calls the previously defined function to execute one training step for each batch.
- `tf.keras.utils.Progbar`: A utility to display a progress bar during training.
- `checkpoint.save(...)`: Saves a checkpoint every 10 epochs.
- Finally, `train(train_data, 50)` starts the training process for 50 epochs.

Section 6: Evaluate Model

This final section evaluates the performance of the trained model on the test dataset.

Cell 96-102: Code - Make Predictions and Evaluate

```

1 # 6. Evaluate Model
2 ## 6.1 Import Metrics
3 from tensorflow.keras.metrics import Precision, Recall
4 ## 6.2 Make Predictions
5 # Get a batch of test data
6 test_input, test_val, y_true = test_data.as_numpy_iterator().next()
7 # Make predictions
8 y_hat = siamese_model.predict([test_input, test_val])
9 # Post processing the results
10 [1 if prediction > 0.5 else 0 for prediction in y_hat ]
11 y_true

```

- `from tensorflow.keras.metrics import Precision, Recall`: Imports metrics that can be used for a more formal evaluation (though they are not used in the subsequent code).
- `test_data.as_numpy_iterator().next()`: Grabs one batch from the test dataset. `y_true` contains the ground-truth labels.
- `y_hat = siamese_model.predict(...)`: Uses the trained `siamese_model` to make predictions on the test batch.
- `[1 if prediction > 0.5 else 0 for prediction in y_hat]`: This is post-processing. The model outputs a probability (a value between 0 and 1). This line converts that probability into a binary prediction: if the probability is greater than 0.5, it's classified as 1 (same person); otherwise, it's 0 (different people).